

INTERVIEW PREPARATION GUIDE

Java Developer navAjna Technologies

Interview Sheet · Core Java · React · Spring Boot

Prepared for Sujith Mondithoka · Walk-in, HITEC City Hyderabad

How to use this guide. Work the sections in order. The plan on the next page is timed — if you fall behind, drop from the bottom, never from the top.

● **Red dot** = high priority, almost certainly asked. ●● = your differentiator. ● **Amber** = worth knowing.

✓ = self-check before moving on. ⚠ = common trap or something to verify.

Contents

Plan	navAjna Technologies — Java Developer Interview Sheet	3
01	Round 1 — Core Java	6
02	Round 1 — Backend Track	16
03	Round 1 — Frontend Track	22
04	Round 2 — Your Projects, and the Hard Questions	29
05	Scenario Questions — Built From Your Own Work	33
06	Walk-In Checklist and One-Page Recall	44

navAjna Technologies — Java Developer Interview Sheet

Sujith Mondithoka · Walk-in, F2F · Karan Arcade, Patrika Nagar, HITEC City, Hyderabad

1. The decision you must make before you walk in

navAjna is hiring for **two separate tracks**, four openings each:

Track	Primary stack	What Round 1 tests
Backend Developer	Java, Spring Boot, Microservices	Core Java + Spring Boot
Frontend Developer	React.js, JavaScript / TypeScript	Core Java + React

Both rounds happen on the same day. Round 1 is technical — core Java plus your position's skills. Round 2 is technical/managerial — project experience, problem solving and fitment.

● My recommendation: go in for the Frontend track

This is the first role in your search where being full-stack is a straight advantage, and the frontend track is where you clear the bar by the widest margin.

Why frontend:

- **The frontend JD's primary skills are your deepest work.** React + Redux, hooks and state management, ES6+/TypeScript, REST integration with JWT, Jest, browser debugging. You have every one of those in production, not in a tutorial.
- **It asks only for "working knowledge of Java"** as the secondary skill — and you have a year of production Spring Boot, JPA and Camunda. **You are not just at that bar, you are well over it.** That is a very comfortable place to be.
- **Your evidence is unusually concrete:** a 14-screen onboarding flow, the My Requests page, Lighthouse 62 → 88 with the actual mechanism, a React 18 / Next.js / TypeScript migration, reusable components reused across 3+ features, Jest and React Testing Library.

Why not backend, even though it is real: on the backend track you would be measured against people whose *only* work is Spring Boot, and the JD adds **Oracle** and **Linux**, which are your two thinnest areas. On the frontend track your Java becomes a bonus rather than the thing being examined.

But this is your call, and the backend case is legitimate — you own two end-to-end Spring Boot applications with Camunda workflows and an AOP audit system. If backend is where you want your career to go, say so and interview for it. Just do not arrive undecided.

What to say at the desk when they ask which position:

"Frontend primarily — React, Redux and TypeScript are my day-to-day. But I have also built the Spring Boot and JPA backends for the same applications, so I can work across both if that is useful to you."

That sentence puts you in the stronger queue while telling them you can fill either gap. With four openings on each side, that flexibility is worth real money to them.

2. You fit this JD better than you may realise

The JD requires	You	Verdict
1-3 years hands-on Java development	~15 months at Zensar + 5 months intern	✓ In band
Notice period 0-7 days — <i>longer will not be considered</i>	Available immediately	✓ Major advantage — see below
Hyderabad, 5 days from office	You are in Hyderabad	✓
CDAC (PG-DAC) or B.E./B.Tech	B.Tech CSE, 8.49 CGPA	✓ The JD accepts engineering graduates
Agile / Scrum	Scrum, Jira, SIT/UAT at Zensar	✓
Good communication, client collaboration	—	● This is being screened for. See §6.

● Your notice period is your single biggest advantage — use it early

The JD says it twice, in bold terms: **"must be able to join within 0-7 days. Candidates with longer notice periods will not be considered."**

Most candidates in a walk-in queue are serving 30, 60 or 90 days. **You can start next week.** For a company staffing eight positions with joining within 7 days, that is not a small detail — it may be the deciding one.

Say it in your first two minutes, in the "tell me about yourself", not at the end:

"...and I am available to join immediately, so the 7-day window is not an issue for me."

The one thing working against you

You left Zensar in **February**, so that is roughly **eight months**. It is the first thing an interviewer will notice and it needs a prepared, calm, two-sentence answer. **File 04, §2.**

3. What navAjna actually is, and why it matters

A Hyderabad product-and-services firm doing **software modernisation and product innovation**. The JD is explicit that you **work across multiple client projects over time** — different domains, codebases and stacks.

Three things that tells you:

1. **They value adaptability over narrow depth.** Someone who has worked both ends of the stack and migrated a platform to React 18 / Next.js / TypeScript fits that story exactly. Say it.
2. **"Interest in learning new technologies" is in the JD.** Have a real answer — what you have learned since February, and how you learn.
3. **It is a smaller company than Infosys or a bank.** The people interviewing you may well be the people you would work with. That makes Round 2 — "fitment" — genuinely about whether they want you on the team. Be someone they would enjoy sitting next to.

4. How to use this sheet

It is deliberately short. You already have the long guides; this is the walk-in version.

Every technical topic also explains where the memory goes, in plain English with a small example — stack versus heap, what an object actually costs, why mutating state does nothing in React, why a forgotten timer is a memory leak. Memory is asked directly in Java interviews, and knowing where things live is what makes the other answers make sense rather than be memorised.

File 01, Part 0 is the foundation — read that first.

Section	Read when
01-core-java.md	Everyone reads this. Round 1 opens here regardless of track.
02-backend-track.md	If you go backend — or to be safe, skim it anyway
03-frontend-track.md	If you take my recommendation, this is your main file
04-round2-and-you.md	Your project stories, the gap, why navAjna, questions to ask
05-scenario-questions.md	22 "what would you do if..." questions built from your own features
06-walk-in-checklist.md	The night before and the morning of

Priority if you are short of time: 01 → 03 (or 02) → 04 → 05.

Sections **04 and 05** matter more than people expect. Round 1 is the one you revise for; **Round 2 is the one that is won or lost**, because it is project experience, problem solving and fitment — and that round is almost entirely scenario questions aimed at what is on your resume.

01 · Round 1 — Core Java

Both tracks start here. The JD asks for "Core Java (OOP, collections, exception handling)" for backend, and "working knowledge of Java" for frontend.

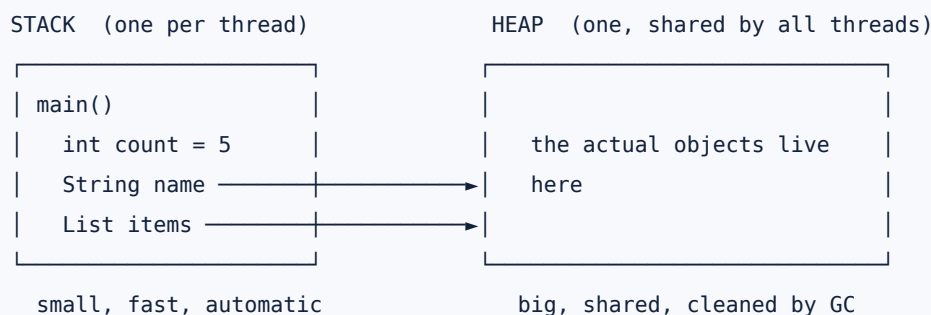
Every topic in this file also explains where the memory goes, in plain English with a small example. Memory questions come up a lot in Java interviews, and more importantly, knowing where things live is what makes the rest of the answers make sense.

Part 0 · How Java uses memory ●●

Read this first. Everything after it refers back here.

The two places things are kept

Java keeps your data in two main places: the **stack** and the **heap**.



The stack holds one box per running method, called a *frame*. Inside the frame are the method's local variables. When the method finishes, the whole frame is thrown away instantly. Each thread gets its own stack, so stack data is never shared.

The heap holds every **object** you create with `new`. There is only one heap and all threads share it. Nothing is removed by hand — the **garbage collector** removes objects that nothing points at any more.

The one rule that explains almost everything

Primitives hold the value. Objects hold a reference — an address pointing into the heap.

```
int count = 5; // the value 5 is IN the stack frame
String name = new String("Sujith"); // the OBJECT is in the heap,
                                     // the stack only holds its address
```

So when you write `int a = 5; int b = a;` you copied the value — two independent 5s. When you write `List a = ...; List b = a;` you copied the *address* — both names point at **one** list, and changing it through either name changes the same object.

The example that makes it click

```
void change(int number, StringBuilder text) {
    number = 99;           // changes only this method's copy
    text.append(" world"); // changes the OBJECT both names point at
}

int n = 5;
StringBuilder s = new StringBuilder("hello");
change(n, s);

System.out.println(n); // 5           - unchanged, we copied the value
System.out.println(s); // hello world - changed, we copied the address
```

Java is always pass-by-value. For objects, the *value being copied is the address*. That is why the first one did not change and the second one did. This is a classic interview question and this example answers it.

The other memory areas, briefly

- **Metaspace** — holds the *class* information: method code, field names, and so on. Loaded once per class, not per object. (Before Java 8 this was called PermGen.)
- **String pool** — a special area of the heap where text literals are kept and reused. More on this below.
- **Static fields** — one copy shared by the whole program, living with the class rather than with any object.

Two errors, and what each one means

Error	What ran out	Usual cause
StackOverflowError	The stack	Recursion with no base case — frames pile up forever
OutOfMemoryError	The heap	Too many live objects — usually something unbounded, like loading a whole table into a list instead of paginating

● Say this if asked about memory generally:

"The stack holds method frames and local variables, one stack per thread, and it is cleaned up automatically when a method returns. The heap holds all the objects and is shared across threads, and the garbage collector removes objects nothing references any more. A primitive variable holds its value directly; an object variable holds a reference into the heap."

Garbage collection in one minute

"The garbage collector removes objects that are no longer **reachable** — nothing in the program points at them any more. Setting a variable to `null` or letting it go out of scope makes the object eligible; it does not delete it immediately. The heap is split into a young area, where new objects go and most die quickly, and an old area for objects that survive. Cleaning the young area is quick and frequent; cleaning the old area is slower."

A **memory leak in Java** is not an unfreed pointer — it is **keeping a reference you no longer need**, usually in a long-lived collection or a static field, so the GC is not allowed to collect it.

Part 1 · OOP

The four pillars, with memory

Pillar	Plain meaning	Your example
Encapsulation	Keep fields private; let people change them only through methods that check the change is allowed	A card application's status goes through a method that verifies the transition is legal
Inheritance	A class reuses another class's code with <code>extends</code>	A base entity holding id and timestamps
Polymorphism	The same call runs different code depending on the real object	A <code>Notifier</code> interface with email and SMS versions; Spring injects the right one
Abstraction	Show <i>what</i> it does, hide <i>how</i>	The controller depends on a service interface, not the implementation

Memory — what an object actually costs:

```
Application app = new Application();
```

`new` allocates space in the **heap** for all the object's fields, plus a small header the JVM uses for its own bookkeeping. The variable `app` sits in the **stack** and holds the address. A subclass object holds its own fields **and** all the fields it inherited, in one block — inheritance does not create two objects.

Memory — where methods live: method *code* is stored once per class in **Metaspace**, not copied into every object. A million objects of one class means a million sets of fields in the heap, but only **one** copy of the code.

Overloading vs overriding — same name with different parameters, decided at compile time · same signature in a subclass, decided at runtime by the real object.

You cannot override a static method. It is *hidden*, and which one runs depends on the reference type, because static methods belong to the class, not to any object.

Abstract class vs interface — abstract class when there is genuinely shared **state and code** (one only); interface for a **contract** (as many as you like).

Memory note: an interface adds no fields, so implementing ten interfaces costs no extra object memory. Extending a class *does* add its fields to every instance.

Why no multiple inheritance of classes? The diamond problem — if two parents override the same method, the compiler cannot pick one. Interfaces are allowed because when two **default** methods clash, the compiler forces you to override and choose.

Part 2 · Collections ●●

Collection

- └ List in order, duplicates allowed → ArrayList, LinkedList
- └ Set no duplicates → HashSet, LinkedHashSet, TreeSet
- └ Queue first in, first out → ArrayDeque, PriorityQueue

Map (NOT a Collection) key → value → HashMap, LinkedHashMap, TreeMap

	Get by index	Search	Order kept	Thread safe
ArrayList	O(1)	O(n)	insertion	no
LinkedList	O(n)	O(n)	insertion	no
HashMap	—	O(1) average	none	no
LinkedHashMap	—	O(1)	insertion	no
TreeMap	—	O(log n)	sorted	no
ConcurrentHashMap	—	O(1)	none	yes, per bucket

● ArrayList vs LinkedList — the memory answer is the whole answer

ArrayList one array, items side by side in memory
[ref][ref][ref][ref][empty][empty]

LinkedList separate node objects, scattered, joined by pointers
[prev|val|next] ↔ [prev|val|next] ↔ [prev|val|next]

Memory: an **ArrayList** is **one array object** in the heap holding references, so it is compact. A **LinkedList** creates a **separate node object for every element**, and each node

stores the value plus a *previous* and a *next* pointer — roughly three times the overhead per item, scattered across the heap.

Why that matters for speed: because an ArrayList's slots sit next to each other, the CPU loads neighbouring items into its cache together, so walking it is fast. LinkedList nodes are in different places, so every step is a fresh memory lookup.

"I use ArrayList by default — O(1) access by index, and the elements sit together in memory which is cache friendly. LinkedList only wins when I am repeatedly inserting or removing in the middle at a position I already hold, and it costs about three times the memory per element because every item is a separate node with two pointers."

How ArrayList grows: it starts with room for 10. When full it creates a **new, bigger array** (about 1.5×), copies everything across, and the old array becomes garbage. So a list that grows to a million items has allocated and thrown away several large arrays along the way — which is why `new ArrayList<>(expectedSize)` matters when you know the size.

●● HashMap internals — including where the memory goes

What `put` does, step by step:

"`put` calls `hashCode()` on the key, then HashMap mixes the bits with `h ^ (h >>> 16)` and picks the bucket with `(n - 1) & hash` — a fast bitwise AND instead of a division, which works because the capacity is always a power of two.

If the bucket is empty it stores the entry there. If not, that is a **collision**, so it walks the entries comparing keys with `equals()` — a matching key replaces the value, otherwise the new entry is added to that bucket. Since Java 8, once one bucket holds more than **8** entries it converts from a linked list to a **red-black tree**, so the worst case drops from O(n) to O(log n).

Default capacity **16**, load factor **0.75**, so at **12** entries it doubles the table and rehashes everything into the new buckets."

Memory — what a HashMap actually holds:

```
table (one array in the heap)
[0] → null
[1] → Node{hash, key, value, next} → Node{...}    ← a collision chain
[2] → null
[3] → Node{hash, key, value, next}
```

Three things cost memory: **one array** for the table, **one Node object per entry** (each holding the hash, a key reference, a value reference and a next pointer), and empty slots — with a 0.75 load factor, roughly a quarter of the table is deliberately empty to keep collisions rare.

Resizing is expensive in memory too: doubling means allocating a new array **twice the size** while the old one still exists, then rehashing every entry. For a moment both arrays

are in the heap.

• equals and hashCode — and why keys should be immutable

"**Equal objects must have equal hash codes.** If I override `equals` but not `hashCode`, two objects that are equal can produce different hashes, land in different buckets, and the map never finds what I stored.

It is also why keys should be immutable. The bucket was chosen from the hash **at insert time**. If a field used in the hash changes afterwards, the entry is stranded in a bucket that no longer matches — the object is still in memory, but unreachable through the map."

Quick answers

- **HashMap vs Hashtable** → Hashtable locks the whole map and is legacy. Use **ConcurrentHashMap**, which locks per bucket so threads writing to different buckets do not block each other.
- **HashSet vs TreeSet** → $O(1)$ unordered vs $O(\log n)$ sorted. **Memory:** a `HashSet` is literally a `HashMap` where every value is the same shared dummy object — so it costs about the same as a map.
- **fail-fast** → `ConcurrentModificationException`, detected with a `modCount` counter. Remove with `iterator.remove()` or `removeIf`, never `list.remove()` inside a for-each.
- **Comparable vs Comparator** → natural order inside the class (one) vs order defined outside it (as many as you want).

Part 3 · Strings and memory ●●

This is where memory questions are asked most often.

Why String is immutable

Once made, a String's characters can never change. `s.concat("x")` returns a **new** String and leaves the original alone.

The four reasons:

1. **The string pool** needs it (below) — sharing is only safe if nobody can change it.
2. **Security** — Strings hold URLs, file paths and credentials. If they could change, a value could be checked and then altered before use.
3. **Thread safety** — something that cannot change is safe to share with no locking.
4. **The hash is cached** — which makes String a fast map key, and that cache is only valid because the value is fixed.

●● The string pool — the question they love

```
String a = "hello";           // goes in the POOL
String b = "hello";           // reuses the SAME pooled object
String c = new String("hello"); // forces a NEW object outside the pool

a == b           // true   - same object
a == c           // false  - different objects
a.equals(c)      // true   - same characters
c.intern() == a  // true   - intern() returns the pooled one
```

HEAP



"Text written as a literal goes into the string pool, and identical literals share one object — so `==` happens to be true. `new String(...)` deliberately creates a separate object in the heap, so `==` is false even though the characters match. That is why I always compare strings with `.equals()` — `==` works by accident until it does not."

● String vs StringBuilder — a memory question disguised as a speed question

```
// BAD - inside a loop
String result = "";
for (String part : parts) {
    result = result + part;    // creates a NEW String object every time
}

// GOOD
StringBuilder sb = new StringBuilder();
for (String part : parts) {
    sb.append(part);          // changes ONE buffer
}
String result = sb.toString();
```

"Because String is immutable, + in a loop cannot change the existing object — it builds a new one each time and throws the old one away. A thousand iterations creates a thousand throwaway objects for the garbage collector, and it is O(n²) because it copies everything each round. StringBuilder keeps one internal character array and appends into it, so it is O(n) and allocates far less."

`StringBuffer` is the same as `StringBuilder` but synchronised — older, slower, rarely needed.

• The Integer cache — a favourite trick question

```
Integer a = 127, b = 127;
Integer c = 128, d = 128;

a == b      // true   - Java keeps a cache of Integer objects from -128 to 127
c == d      // false  - outside the cache, so two separate objects
c.equals(d) // true
```

"Java pre-creates Integer objects for -128 to 127 and reuses them, so `==` is true in that range. Above it, autoboxing creates a new object each time, so `==` compares two different addresses. Always `.equals()` for boxed types."

Memory point worth adding: `int` is a primitive held in the stack; `Integer` is an object in the heap with a reference pointing at it. So `List<Integer>` of a million numbers is a million small heap objects, not a compact block of ints — which is why `int[]` is far lighter than `List<Integer>`.

Part 4 · Exception handling

```
Throwable
├── Error                JVM-level trouble – do not catch (OutOfMemoryError)
├── Exception
│   ├── RuntimeException  UNCHECKED – NullPointerException, IllegalArgumentException
│   └── everything else   CHECKED   – IOException, SQLException
```

- **Checked** = the compiler makes you catch it or declare `throws`. **Unchecked** = usually a bug in the code.
- **finally** **always runs**, except if the JVM exits. **Never return from finally** — it swallows the real return value and any exception.
- **try-with-resources** closes anything `AutoCloseable` automatically, even when an exception is thrown.
- `throw` raises one now; `throws` declares that a method might.

Memory: an exception is an object in the heap like any other, and creating one captures the **stack trace** — a snapshot of every frame on the stack at that moment. That is why exceptions are not free, and why using them for ordinary control flow in a loop is slow.

● **Connect it to Spring:**

"I use unchecked custom exceptions with a shared `BusinessException` parent, handled centrally by `@RestControllerAdvice` so the API returns one error shape. Unchecked matters for `@Transactional` too, because by default Spring only rolls back on unchecked exceptions."

Part 5 · Java 8 — streams and lambdas

```
Map<Status, Long> byStatus = apps.stream()
    .filter(a -> a.getAmount() > 1000)
    .collect(Collectors.groupingBy(Application::getStatus, Collectors.counting()));
```

- **Intermediate operations are lazy** (`filter` , `map` , `sorted`); **terminal operations trigger the work** (`collect` , `forEach` , `reduce` , `findFirst`). A stream is used once.
- **map vs flatMap** — one-to-one vs one-to-many then flattened into a single stream.
- **The four interfaces:** `Predicate` →filter, `Function` →map, `Consumer` →forEach, `Supplier` →a value made only when needed. One abstract method each.
- **Optional** — use `map` , `orElse` , `orElseThrow` ; not `isPresent` + `get` .

Memory — why laziness matters:

"Because the intermediate steps are lazy, the stream does not build an intermediate list at each stage. `filter` then `map` passes each element through both steps one at a time, so only one element is in flight. If each stage built its own list, a million-row stream with three stages would allocate three million-element lists."

⚠ `sorted()` is the exception — it **must** hold everything in memory to sort it. And `collect(toList())` builds the final list, which is the point.

Memory — lambdas: a lambda that uses no outside variables can be reused by the JVM rather than created fresh each time. A lambda that **captures** a variable has to hold onto it, which is why captured local variables must be effectively final — the lambda keeps a copy, and letting it change would be ambiguous.

Part 6 · The likely coding question

Expect **one** easy problem: reverse a string, palindrome, first non-repeating character, word frequency, find duplicates, second largest.

```
// first non-repeating character
Map<Character, Integer> counts = new LinkedHashMap<>(); // LinkedHashMap keeps order
for (char c : s.toCharArray()) counts.merge(c, 1, Integer::sum);
for (var e : counts.entrySet()) if (e.getValue() == 1) return e.getKey();
return null;
```

Memory to mention: $O(k)$ extra space, where k is the number of distinct characters — the map holds one entry per distinct character, not per character in the string.

```
// second largest - no sorting, no extra memory
int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
for (int n : nums) {
    if (n > first) { second = first; first = n; }
    else if (n > second && n != first) second = n;
}
```

Memory to mention: $O(1)$ extra space — two numbers, whatever the array size. Sorting would be $O(n \log n)$ time and may copy the array.

The four free marks almost everyone drops

1. **Ask the edge cases first** — null, empty, duplicates, upper/lower case.
2. **Say the brute force and its cost**, then improve it. *"Two nested loops is $O(n^2)$ — a HashMap makes the lookup $O(1)$, so one pass."*
3. **Talk while you write**. Silence looks like being stuck.
4. **State time AND space at the end**, before they ask. Most candidates give only time.

✓ Before you walk in

1. Stack vs heap, and the `change(int, StringBuilder)` example that proves pass-by-value.
2. HashMap `put` end to end, plus what it holds in memory — table array, one Node per entry.
3. The string pool, with the four lines of `a == b` / `a == c`.
4. Why `+` in a loop is slow — new object each time, $O(n^2)$, garbage.
5. The Integer cache, -128 to 127 .

02 · Round 1 — Backend Track

Skim this even if you go frontend — Round 1 includes core Java either way, and Round 2 will cover your Spring Boot work.

The JD asks for: **Core Java · Spring, Spring Boot, Hibernate/JPA · REST APIs and microservices · SQL, preferably Oracle · JUnit and a mocking framework such as Mockito · comfortable in Linux.**

Two of those are your thin areas — Oracle and Linux. They are at the end of this file with honest answers. Everything before that you can speak to from real work.

| Spring Boot

IoC and DI in one line → "Inversion of Control means the framework creates and wires the objects instead of my class doing `new`. Dependency injection is the mechanism. The benefit is my class depends on an interface, so I can swap the implementation and inject mocks in tests."

- **Constructor injection, and why** → fields can be `final`, dependencies cannot be missing, and you can test with `new Service(mock)` with no Spring context at all. A constructor with eight parameters also makes bad design visible, which field injection hides.

Annotations: `@SpringBootApplication` (= `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`) · `@RestController` (= `@Controller` + `@ResponseBody`) · `@Service` · `@Repository` (also translates DB exceptions into Spring's hierarchy) · `@Autowired` · `@Qualifier` · `@Value` · `@Transactional` · `@RestControllerAdvice`.

REST mapping: `@GetMapping` / `@PostMapping` / `@PutMapping` / `@DeleteMapping`, `@PathVariable` (from the URL), `@RequestParam` (query string), `@RequestBody` (JSON body), `@Valid` (triggers Bean Validation before your method runs).

- **Bean scopes** → **singleton is the default**, one instance for the whole context. So **keep beans stateless** — a mutable instance field is a concurrency bug, and on a banking platform an audit failure, because two requests overwrite each other's value. Request data belongs in parameters, not fields.

Memory — why singleton is the default: one `@Service` object sits in the heap for the whole application's life, however many requests arrive. A thousand concurrent requests share that one object; what is *not* shared is each request's **stack**, which holds its own local variables. That is exactly why locals are safe and instance fields are not.

```

@Service
public class ApplicationService {
    private String currentUser;           // ❌ ONE field, shared by every request

    public void approve(String user) {    // ✓ a parameter lives in THIS
request's stack frame
        ...
    }
}

```

Two requests running `approve` at the same moment each get their own stack frame, so the parameter is private to each. The field is one piece of heap memory they both write to — and the second one wins.

Global exception handling — worth volunteering:

```

@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ApplicationNotFoundException.class)
    public ResponseEntity<ErrorResponse> notFound(ApplicationNotFoundException ex) {
        return ResponseEntity.status(404).body(new ErrorResponse("NOT_FOUND", ex.getMessage()));
    }
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> all(Exception ex) {
        log.error("Unexpected", ex);           // log the detail...
        return ResponseEntity.status(500)
            .body(new ErrorResponse("INTERNAL_ERROR", "Something went wrong")); // ...leak nothing
    }
}

```

"One consistent error shape for the frontend, and a stack trace never reaches the client — it tells an attacker your framework versions and internal structure."

Spring MVC / DispatcherServlet → front controller: every request hits the `DispatcherServlet`, handler mapping picks the controller method, a handler adapter invokes it and binds arguments, a message converter (Jackson) turns the return value into JSON. *Practical value: a 404 is handler mapping, bad JSON is the converter, a binding failure is the adapter.*

Auto-configuration → `@ConditionalOnClass` and `@ConditionalOnMissingBean`. Boot configures a `DataSource` if a driver is on the classpath and you have not defined one — and the moment you define your own bean, yours wins.

JPA and Hibernate

JPA is the specification, **Hibernate** the implementation, **Spring Data JPA** the layer that generates repositories from method names.

Fetch defaults — asked often: `@ManyToOne` and `@OneToOne` are **EAGER**; `@OneToMany` and `@ManyToMany` are **LAZY**. "I set `@ManyToOne` to LAZY explicitly, because otherwise loading a list of applications drags every related entity across even when the screen shows three columns."

● N+1 — the classic question

"One query loads the list, then each row's relation loads with its own query — 100 rows becomes 101 round trips. I find it by turning on SQL logging and counting queries; the same query repeating with a different id is the signature. Fixes: a **JOIN FETCH**, an `@EntityGraph`, **batch fetching**, or a **DTO projection** for a read-only screen, which selects only the columns needed and skips loading entities entirely."

N+1 costs memory as well as time, which few candidates mention: each of those 101 queries returns entities that go into the persistence context with their snapshots. So you are not only paying 101 network round trips, you are filling the heap with objects the screen may only need three fields from.

`@Transactional` → Spring wraps the bean in a **proxy** that opens a transaction before the method and commits after. Two things to know:

- It rolls back on **unchecked** exceptions only, unless you set `rollbackFor`.
- ● **An internal `this.method()` call bypasses the proxy, so no transaction starts.** The same is true of your AOP aspect — a strong point to make, because you built one.

Dirty checking → a managed entity needs no `save()`; Hibernate compares it with the loaded snapshot at flush time and updates only changed columns.

Memory — the persistence context is a cache in your heap. For dirty checking to work, Hibernate keeps **two copies** of every entity you load inside the transaction: the entity itself and a snapshot of how it looked when loaded. That is the *first-level cache*, and it is always on.

Two consequences worth saying out loud:

- Loading 100,000 rows in one transaction puts 200,000 objects in the heap, and that is a very common cause of `OutOfMemoryError` in batch jobs. The fixes are paging, or clearing the persistence context periodically.
- A **DTO projection** avoids all of it — it selects only the columns you need and never puts managed entities in the context, so there is no snapshot and no dirty checking. That is why it is the right choice for a read-only screen or report.

Details you can claim from your own work: `@Enumerated(EnumType.STRING)` not the ordinal default · `BigDecimal` for money, never `double` · **Liquibase** for versioned schema migrations, so a schema change is reviewable and repeatable across SIT, UAT and production.

| REST and microservices

REST design → nouns not verbs (`/applications/{id}/approve` as a sub-resource), correct status codes (201 + Location on create, 204 on delete, 400 vs 401 vs 403 vs 404, 409 for a conflict), **pagination on every list endpoint**, URL versioning, **DTOs at the boundary not**

entities — returning entities leaks your schema and causes lazy-loading surprises during serialisation.

● **Microservices — answer honestly about depth**

"On the Service Online platform I built Spring Boot services and REST APIs consumed by other parts of the platform and by the React frontend, and the platform used Kafka and RabbitMQ between components. What I have not done is own the decomposition — deciding service boundaries, standing up discovery and a gateway.

The way I think about it: microservices buy independent deployment, scaling and fault isolation, and you pay with network failure, no distributed transaction and harder debugging. For a small system a well-structured monolith is usually the better call."

Saying the cost unprompted is what scores. Most candidates recite only the benefits.

Resilience, if pushed: always a timeout; a **circuit breaker** (closed → open → half-open) so a slow dependency does not exhaust your threads; retry **only idempotent** operations.

| **JUnit and Mockito — named in the JD**

```
@ExtendWith(MockitoExtension.class)
class ApplicationServiceTest {
    @Mock ApplicationRepository repository;
    @Mock NotificationSender notifier;
    @InjectMocks ApplicationService service;

    @Test
    void notifiesOnApproval() {
        when(repository.findById(1L)).thenReturn(Optional.of(new Application(1L, PENDING)));

        service.approve(1L, "authoriser-7");

        ArgumentCaptor<Application> saved = ArgumentCaptor.forClass(Application.class);
        verify(repository).save(saved.capture());
        assertEquals(APPROVED, saved.getValue().getStatus());
        verify(notifier).send(eq("authoriser-7"), any(Message.class));
    }
}
```

- **@Mock** creates the double · **@InjectMocks** builds the class under test with mocks injected · **@Spy** wraps a real object · **@MockBean** is the Spring-context version.
- **when().thenReturn()** stubs · **verify()** asserts an interaction happened · **ArgumentCaptor** asserts on **what** was passed.
- **Void methods:** you cannot assert a return, so **verify(...)** . To make one throw you need **doThrow(ex).when(mock).method()** — **when(mock.voidMethod())** will not compile.

- **Coverage** tells you what *ran*, not what was *verified* — a test with no assertion still counts.
- **Be honest if asked how much Mockito you have used.** Your resume lists JUnit, Jest and React Testing Library. *"I have written JUnit tests for REST APIs and workflow logic validated through SIT and UAT, and Jest and RTL on the frontend. My Mockito usage has been lighter than my JUnit usage, so I would not overstate it — but I understand the model."* Then show the example above.

| • SQL — and the Oracle question

Your databases are **PostgreSQL and MySQL**; the JD says *"preferably Oracle"*. That is a preference, not a requirement, and **SQL skill transfers almost entirely**. Say so plainly:

"My production work has been PostgreSQL, with MySQL before that. I have not used Oracle specifically, but the SQL is largely the same — joins, indexing, execution plans and query tuning all transfer. The differences I would expect are dialect-level: sequences rather than auto-increment, `ROWNUM` or `FETCH FIRST` instead of `LIMIT`, `NVL` instead of `COALESCE`, and `SELECT ... FROM DUAL` for a scalar. I would be productive quickly."

That answer is much better than pretending. Knowing the four specific differences proves you actually understand what varies, which is the real question.

Core SQL to have ready:

- **Joins** — INNER (matches only), LEFT (all from the left, nulls on the right). *"Rows with no match"* = `LEFT JOIN ... WHERE right.id IS NULL`. ⚠️ A condition on the right table in the `WHERE` of a LEFT JOIN silently makes it an INNER JOIN — it belongs in the `ON`.
- **WHERE vs HAVING** — rows before grouping vs groups after aggregation.
- **Indexing** — a B-tree mapping values to rows; turns a full scan into $O(\log n)$. **Always say the cost:** every insert and update maintains every index, so you add them deliberately. **Left-most prefix rule** — an index on `(status, created_at)` does not serve `created_at` alone. Index killers: a **function on the column**, a **leading %** in LIKE.
- **EXPLAIN** — `type: ALL` means full scan; check which index was used and the row estimate.
- **ACID** — the transfer example: debit and credit must be atomic or money disappears.

| • Linux — the JD asks for "working comfortably in Linux"

Be honest about level, then show the working set. Most of this you have done via Docker, Git and deployments.

```
ls -la · cd · pwd · cp · mv · rm -rf · mkdir -p · find . -name "*.log"
cat · less · head -50 · tail -f app.log          # tail -f is THE one for live logs
grep -i "error" app.log · grep -rn "ClassName" . · grep -c "Exception" app.log
ps -ef | grep java · kill -9 <pid> · top · df -h · free -m
chmod 755 run.sh · chown · export VAR=value
scp file user@host:/path · ssh user@host
java -jar app.jar · tail -f nohup.out
```

The answer that sounds like real use, not a cheat sheet:

"Comfortable at the level a developer needs — navigating, finding things and reading logs. The commands I actually live in are `tail -f` on an application log while reproducing an issue, `grep` to find an exception or a string across a codebase, `ps -ef | grep java` to check what is running, and `df -h` when something fails for no obvious reason and the disk turns out to be full. I would not call myself a systems administrator."

One pipeline worth knowing — it looks good and is genuinely useful:

```
grep "ERROR" app.log | awk '{print $5}' | sort | uniq -c | sort -rn | head
# the most frequent error types, most common first
```

✓ Before you walk in

1. HashMap internals, and the N+1 answer with four fixes.
2. Why `@Transactional` does nothing on an internal call — and that your AOP aspect has the same proxy caveat.
3. The microservices answer: your level, then the **costs**.
4. The Oracle answer, with the four dialect differences.
5. The Linux answer, with `tail -f`, `grep`, `ps -ef`.

03 · Round 1 — Frontend Track

The track I would take. Your strongest ground.

The JD asks for: **React.js and Redux** (components, hooks, state management) · **ES6+ / TypeScript, HTML5, CSS3** · **REST integration including JWT authorisation** · **Jest unit testing** · **browser-based debugging** · **working knowledge of Java**.

You have every one of those in production. This file is the compressed version — answers, not explanations.

React — the questions that will come

Why React? *"It is declarative — I describe what the UI should look like for a given state and React works out the DOM changes, instead of me keeping the DOM and the data in sync by hand. Plus the component model: I built reusable components and Redux patterns used across three or more features on the platform."*

Virtual DOM → a JS object tree; React diffs the new tree against the old (reconciliation) and applies the minimum real DOM changes, because DOM operations trigger layout and paint.

Memory — JavaScript keeps things in a stack and a heap too, the same split as Java:

```
let count = 5;           // a number - the value sits in the stack
let user = { name: 'Sujith' }; // the OBJECT is in the heap,
                           // `user` holds a reference to it
```

Primitives (number, string, boolean, null, undefined) hold their value. Objects, arrays and **functions** are heap values, and the variable holds a reference. That single fact explains three React behaviours at once:

- **Why mutating state does nothing.** `items.push(x)` changes the object in the heap but the reference is unchanged, so React's comparison sees the same address and skips the render. `[...items, x]` allocates a **new** array at a **new** address, which is what React detects.
- **Why `useCallback` exists.** A function is a heap object. Every render creates a *new* one at a *new* address, even though the code is identical — so a `React.memo` child sees a "changed" prop. `useCallback` keeps the same object.
- **What the Virtual DOM is.** A tree of plain JS objects in the heap — cheap to build and throw away, which is the whole reason the approach works.
- **key and why index is dangerous** → A key is a stable identity across renders. With `key={index}`, deleting the first item makes React think the item at index 0 just changed its props, so it **reuses the DOM node** — a typed input value stays on the wrong row. With `key={item.id}` React knows that item is gone and unmounts it. Index is fine only for a static list that is never reordered or filtered.

Props vs state → in from the parent, read-only · owned here, changed via the setter. Data flows down, events flow up through callbacks. Two siblings need it → **lift the state up**.

● What causes a re-render, and how you stop unnecessary ones

Own state changed · props changed · **the parent re-rendered** (even with identical props) · context value changed.

Fixes, in order: `React.memo` → `useCallback` / `useMemo` on the props you pass it → **move state down** → split contexts.

Your answer, and it is a strong one: "On the platform I took Lighthouse from 62 to 88. Two of the three things were render-related: route-based code-splitting with `React.lazy`, so each module's JS only downloads when opened, and memoising the Redux selectors, because unmemoised selectors returned a new array each call, so every connected component saw a changed prop and re-rendered on unrelated state changes."

That is the best technical story you have for this interview. Have it ready.

Hooks

Hook	For
<code>useState</code>	local state
<code>useEffect</code>	sync with the outside — fetch, timers, subscriptions
<code>useRef</code>	a value that persists without re-rendering; DOM access
<code>useMemo</code> / <code>useCallback</code>	cache a value / a function reference
<code>useContext</code>	read shared data without prop drilling
<code>useReducer</code>	complex state with many transitions

Rules of hooks → top level only, never in a condition or loop. **Why:** React tracks hooks **by call order**, so a skipped hook shifts the order and you get the wrong state.

`useEffect` **dependency array** → no array = every render · `[]` = once on mount · `[x]` = when x changes. The `return` is **cleanup** — clear timers, remove listeners, abort fetches, or you leak and get "setState on an unmounted component".

Memory — this is what a leak in React actually is. JavaScript frees an object when nothing references it any more. A timer, an event listener or an open request **still holds a reference to your component's function and everything it closed over** — so the component is removed from the screen but cannot be freed.

```
useEffect(() => {
  const id = setInterval(tick, 1000);
  window.addEventListener('resize', onResize);

  return () => { // without this, both keep the component alive
    clearInterval(id);
    window.removeEventListener('resize', onResize);
  };
}, []);
```

A page the user opens and closes fifty times then holds fifty live components in memory. That is the real reason cleanup matters — the console warning is just the symptom.

Memory — closures. A closure keeps the whole scope it captured alive. That is what makes `useState` and debounce work, and it is also why a forgotten listener holds on to so much.

● **When NOT to use `useEffect`** → for anything derivable. `const filtered = items.filter(...)` during render beats storing it in state and syncing it in an effect.

Three `useState` traps: updates are batched so `setCount(c => c + 1)` when the new value depends on the old · **never mutate** — `items.push()` keeps the same reference so React skips the render; use `[...items, x]` · `useState(() => expensive())` to run the initialiser once.

Controlled vs uncontrolled → React state is the source of truth vs the DOM holds it.

"I default to controlled — it makes validation and disabling the submit button straightforward. On the 14-screen Business Card flow I used React Hook Form, which keeps fields uncontrolled for performance so the whole form does not re-render on every keystroke."

Redux

Why Redux and not just Context? → "Context solves prop drilling but every consumer re-renders when the value changes, and it does no caching. Redux gives one store, predictable updates through reducers, and the DevTools action log — which matters when several distant parts of the app read and write the same state."

The flow: `dispatch(action)` → reducer (pure function: old state + action → new state) → store updates → connected components re-read.

Redux Toolkit → `createSlice` collapses action types, creators and the reducer into one place. ● **"Why can you write `state.items.push()` in RTK?"** → "Immer. You mutate a draft and Immer produces a correctly copied new state, so it is still immutable — it just removes the spread operators."

● **Memoised selectors** → `createSelector` from Reselect only recomputes when its inputs change and returns the same reference otherwise. **This is your Lighthouse story — be ready to explain it, because you listed it.**

Memory — say it this way, it is the clearest version of your story:

```
// ❌ allocates a NEW array in the heap on every call
const selectPending = state => state.apps.filter(a => a.status === 'PENDING');

// <span class="tick">✓</span> returns the SAME array reference until state.apps actually changes
const selectPending = createSelector([state => state.apps],
  apps => apps.filter(a => a.status === 'PENDING'));
```

"The unmemoised version returns a new array every time it runs. React compares props by reference, not by contents — so even when the data was identical, the address had changed, every connected component saw a new prop and re-rendered on unrelated state changes. Memoising it meant the same reference came back until the input genuinely changed."

Trade-off to mention: memoisation is not free — it stores the last inputs and the last result, so it trades a little memory for avoided work. Worth it on a hot path, not worth it everywhere.

JavaScript / TypeScript

- **var vs let / const** → function vs block scope; **const** prevents reassignment, not mutation of the object.
- **Closure** → a function that remembers the scope it was created in. It is how **useState** and **debounce** both work.
- **Event loop** → one call stack; slow work goes to the browser, callbacks queue, and **microtasks (Promises) drain fully before any macrotask (setTimeout)**.
`console.log('A'); setTimeout(...'B'); Promise.then(...'C'); console.log('D')` → **A D C B**.
- **== vs ===** → always **===**. **?? vs ||** → **||** replaces any falsy value, so `0 || 10` is 10; **??** only replaces null/undefined.
- **Spread is a shallow copy** — nested objects are still shared.
- **TypeScript, why** → "It catches a whole class of bugs before the code runs, especially around API response shapes, and it makes refactoring safe because the compiler shows every place that breaks. I was part of the platform's migration to React 18, Next.js and TypeScript."
- **interface vs type** → both describe objects; only **type** does unions. **any** disables checking, **unknown** forces you to narrow first.

• REST integration and JWT — named explicitly in the JD

```
const [data, setData] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  const controller = new AbortController();
  (async () => {
    try {
      setLoading(true); setError(null);
      const res = await fetch('/api/applications', {
        headers: { Authorization: `Bearer ${token}` },
        signal: controller.signal,
      });
      if (!res.ok) throw new Error(`HTTP ${res.status}`); // fetch does NOT throw on 4xx/5xx
      setData(await res.json());
    } catch (e) {
      if (e.name !== 'AbortError') setError(e.message);
    } finally { setLoading(false); }
  })();
  return () => controller.abort(); // cleanup: cancel on unmount
}, []);
```

Three things to point out unprompted:

1. **res.ok** — **fetch** resolves on 404 and 500; only a network failure rejects. You throw.
2. **AbortController in the cleanup** — stops the stale-response race condition *and* the unmounted-component warning.
3. **The four states: loading, error, empty, success.** *Empty* is the one people forget — "No applications match" beats a blank area.

JWT → **header.payload.signature**. **Signed, not encrypted** — anyone can base64-decode the payload, so nothing sensitive goes in it. Sent as **Authorization: Bearer <token>**. The server validates the signature and expiry, so no session lookup is needed.

• **Storage:** an **httpOnly** cookie is safer than **localStorage**, because JavaScript cannot read it, so an injected script cannot steal it. **And authorisation must be enforced server-side** — hiding a button in the UI is not security.

401 vs 403 → not authenticated vs authenticated but not allowed. Handle 401 in one place (an axios interceptor) and redirect to login.

Jest and testing — named in the JD

```
test('shows an error when the reference is blank', async () => {
  render(<ApplicationForm />);
  await userEvent.click(screen.getByRole('button', { name: /submit/i }));
  expect(await screen.findByText(/reference is required/i)).toBeInTheDocument();
});
```

The principle to state: *"React Testing Library makes you test what the user sees and does — query by role and label, not by class name. The test then survives a refactor: if I rewrite the internals but the behaviour is the same, it still passes."*

Note the accessibility overlap — if you cannot query by role or label, the markup probably is not accessible. **You built WCAG 2.1-compliant notification preferences, so you can say that with a straight face.**

`jest.fn()` for a mock function, `jest.mock('./api')` for a module, `screen.findBy*` (async) vs `getBy*` (sync) vs `queryBy*` (returns null, for asserting absence).

Browser debugging — the JD asks for this specifically

- **Elements** — inspect the DOM and computed styles; check what CSS is actually winning.
- **Console** — errors and logs.
- **Network** — status codes, payloads, timing, and whether a call even fired. **Throttle to Slow 4G** to see what a real user sees.
- **Performance** — record an interaction, find long tasks and layout thrashing.
- **React DevTools Profiler** — *"this is how I found the unnecessary re-renders behind the Lighthouse score."*
- **Sources** — breakpoints, step through, inspect scope. Better than `console.log` once you are past a trivial bug.

Core Web Vitals, in case performance comes up: **LCP < 2.5s** (is it there) · **CLS < 0.1** (does it stay still — set image dimensions) · **INP < 200ms** (does it respond).

HTML / CSS quick answers

- **Semantic HTML** → `<header>` `<nav>` `<main>` `<button>` — accessibility, SEO, readability. A `<button>` is keyboard-focusable and announced correctly; a `<div onClick>` is not.
- **Flexbox vs Grid** → one dimension vs two. `grid-template-columns: repeat(auto-fit, minmax(260px, 1fr))` is a responsive grid with no media queries.
- **box-sizing: border-box** → width includes padding and border, which is what you expect.
- **Responsive** → mobile-first with `min-width` queries, the viewport meta tag, relative units, `clamp()` for fluid type.
- **Centre** → `display: grid; place-items: center`.

✓ Before you walk in

1. The Lighthouse 62 → 88 story **with the mechanism** — code-splitting, `React.lazy`, memoised selectors. In under 60 seconds.
2. The index-as-key failure, with the delete example.
3. The four states, and `res.ok`.
4. What a JWT is, and why it is not secret.
5. The four causes of a re-render and three ways to reduce them.

04 · Round 2 — Your Projects, and the Hard Questions

Round 2 is explicitly "project experience, problem solving and fitment". At a company of this size, the people across the table are likely the people you would work with — so this round is genuinely about whether they want you on the team.

| • "Tell me about yourself" — your opening 90 seconds

Lead with the stack, land on availability.

"I am a software engineer with about a year and a half of production experience, most recently at Zensar on Standard Bank South Africa's Service Online platform — an enterprise banking platform used by around 20,000 corporate clients.

I worked full-stack on it. On the frontend that was React, Redux and TypeScript — I built a 14-screen onboarding flow, the My Requests page, and the notification feed, and I took the Lighthouse score from 62 to 88 with code-splitting and selector memoisation. On the backend I built the Spring Boot, JPA and PostgreSQL services behind those screens, including a Camunda approval workflow and an AOP audit system.

Frontend is where my depth is, but I have shipped both, which is why this role interested me — and **I am available to join immediately**, so the seven-day window is not a problem."

Practise this twice tonight. Time it. The last line is the one they are listening for.

| Your four stories

Each: **what it was → a decision you made → what was hard → what you would do differently.** That last part is what makes an answer sound senior rather than junior.

Story 1 • — Lighthouse 62 → 88 (lead with this on the frontend track)

"The platform's pages were slow. I profiled with Lighthouse and React DevTools rather than guessing. Two things stood out. We were shipping every module in the initial bundle, so I added **route-based code-splitting with `React.lazy`** — each module's JavaScript only downloads when it is opened. And the **Redux selectors were not memoised**, so they returned a new array on every call; every connected component saw a changed prop and re-rendered on completely unrelated state changes. Memoising them stopped that.

62 to 88. What I took from it is to measure first — my first guess about the cause was wrong."

That last sentence is worth saying. It shows method, not luck.

Story 2 ● — The AOP audit system (*lead with this on the backend track*)

"We needed every report download and email captured for compliance. The obvious approach is a logging call inside each endpoint, but that is dozens of files and easy for the next person to forget. So I used Spring AOP — a pointcut matches the methods and the aspect runs around them, so the audit logic lives in one class and the business code is untouched. It records 50,000+ transactions a day to PostgreSQL."

Follow-up to have ready: *pointcut* selects which methods, *advice* is the code that runs (`@Before` , `@After` , `@Around`). Spring uses a **proxy**, so — like `@Transactional` — an internal `this.method()` call does not trigger the aspect. Saying that unprompted is strong.

Story 3 — Business Card, owned end to end

A 14-screen React onboarding flow with React Hook Form and **two-layer validation**, plus the Spring Boot / JPA / PostgreSQL backend and a Camunda BPMN approval workflow. **60% off card processing time.**

The point to make about the two layers: *"Client-side validation is for the user's convenience — it is not a security control, because anyone can bypass the browser and post straight to the API. So the same rules are enforced server-side."* In a banking context that lands well.

On the multi-step form: all the form state lives in the parent, so moving backwards never destroys what was entered. React Hook Form keeps fields uncontrolled, so the whole form does not re-render on every keystroke.

Story 4 — Recall Transactions, and accessibility

7 accessible multi-step screens with **navigation guards**, Spring Boot REST APIs, JPA and a Camunda process, replacing a branch-only paper process.

"A recall is a financial instruction, so a user accidentally hitting back or closing the tab midway was a real problem — hence the guards. And the screens were built accessible: labels tied to inputs, keyboard navigation, errors announced rather than only shown in red. I did the same for the notification preferences, which were WCAG 2.1 compliant."

●● The three questions you must not fumble

1. The eight-month gap — prepare this above everything else

You left Zensar in **February**. It is now **October**. That is roughly **eight months**, and it is the first thing an interviewer will notice.

Two or three sentences. Factual. Calm. No apology.

"[The real reason — the project rolled off / the engagement ended / a personal reason.] Since then I have been [specifically what you did]. I was deliberate about wanting a hands-on product role rather than taking the next thing available, and I am available to start immediately."

Fill in the truth — do not invent anything. Project roll-offs are routine in the services industry and nobody holds one against you.

But at eight months, "I was upskilling" is not enough. Be specific and have evidence:

- The **Crio.Do Software Development Fellowship** is on your resume as ongoing — say what you have built or completed in it.
- Anything on GitHub since February. Have it ready to show — this is a **face-to-face** interview, so you can literally open it.
- The GenAI Level 1 and 2 certifications.

Then pivot to availability, which turns the liability into the asset:

"It does mean I can join inside your seven-day window, which I know matters for this role."

2. "You look full-stack. Which do you actually want?"

Do not hedge — hedging reads as not knowing what you want.

"Frontend is where my depth is — React, Redux and TypeScript are what I work in daily, and the performance and component work is what I enjoy most. But I have built and owned the Spring Boot backends for the same applications, so I am comfortable across the stack. For this role I would want to be on the frontend side, while being useful on backend tickets when the team needs it."

(Swap the emphasis if you decide on backend — but decide.)

3. "Why navAjna?"

Use their own words back at them — the JD talks about working across multiple client projects.

"Two reasons. The JD says you work across multiple client projects over time, with different domains and stacks — that suits me, because I have already worked both ends of a stack and been through a migration to React 18, Next.js and TypeScript. I would rather keep learning new codebases than do the same thing for three years.

And it is a smaller team than my last project, where I was one of many on a large platform. I want work where I own features end to end and see the impact — which I did on Business Card and Recall, and that was the most satisfying part of the job."

Do not say "it is a good opportunity" or "for growth".

• Communication is being screened for

The hiring brief for this role says candidates should be **smart in communication skills**, and the JD asks for *"good communication skills and the ability to collaborate with team members and client stakeholders."*

Four things that make the difference in a face-to-face:

1. **Structure your answers.** "Three things —" then actually give three. It sounds organised and it stops you rambling.
2. **Finish your sentences and stop.** Trailing off and over-explaining is the most common nervous habit. Say your answer, then be quiet.
3. **If you do not know something, say so clearly** — *"I have not used that directly. My understanding is X — is that the direction you mean?"* That has never cost anyone an offer. Bluffing has.
4. **Ask questions back.** It makes it a conversation rather than an interrogation, and in a fitment round that is the whole point.

Questions to ask them

Have four, ask two or three. Ending with "no, I'm good" reads as no interest.

1. "Which client project would this role start on, and what is the stack there?"
2. "How is the team structured — would I own features end to end, or work on parts of a larger piece?"
3. "What does the code review and release process look like?"
4. "The JD mentions working across multiple client projects — how often does that rotation happen in practice?"
5. **The closing one:** "What would a successful first three months look like in this role?"

Then **listen**, and connect your answer back to it.

Logistics

Notice period	Immediately available. Say it early and clearly — the JD rules out longer notice periods, so this is your edge.
Location	Hyderabad, 5 days from office. You are local. Say so.
Salary	Know your last fixed CTC and your target before you walk in. Never inflate — it is verified against payslips. Ask for the fixed/variable split at offer stage.
Education	B.Tech is explicitly accepted by the JD. You are not disadvantaged by not being CDAC.

05 · Scenario Questions — Built From Your Own Work

Round 2 is "project experience, problem solving and fitment". That round is mostly scenario questions, and they will be aimed at what is on your resume — because that is the only thing the interviewer knows about you.

Every scenario below is one they can plausibly build from your CV. The answers are grounded in what you actually did on Service Online.

How to answer ANY scenario question — the five steps

Most candidates jump straight to a fix. The marks are in the method.

1. CLARIFY One or two questions. "Is it slow for everyone, or one client?"
2. DIAGNOSE Say how you would FIND OUT before saying what you would change.
3. CAUSES Give the likely causes, most likely first.
4. FIX + COST The fix, and what it trades away. Nothing is free.
5. PREVENT What you would add so it does not happen again.

Step 5 is the one almost nobody does, and it is what separates someone who fixes tickets from someone you would trust with a service. A test, a log line, an alert, a validation.

The sentence to reuse all day: *"Before I change anything I would want to know where the time is actually going — my first guess is usually wrong."* You have earned the right to say that: on the Lighthouse work your first guess *was* wrong, and measuring is what found it.

Part A · Frontend scenarios

S1 • "Your My Requests page now has 500 applications and takes 6 seconds to load. Fix it."

Clarify: Is the 6 seconds the API call, or the rendering? Is the data paginated server-side or do we fetch everything?

Diagnose: *"The Network tab tells me whether the time is the request or the render. The React DevTools Profiler tells me how long the commit takes and how many components rendered."*

Causes, most likely first:

1. The API returns all 500 rows — the payload itself is the cost.
2. All 500 rows render at once, so 500 components mount.
3. Re-rendering the whole list on every filter or tab change.

Fix, with the trade-off:

"First choice is **server-side pagination** — the API returns 20 rows and a page count. It is the biggest win because it fixes the payload *and* the render, and it keeps working as the data grows. The cost is a round trip per page, which I would soften by keeping the previous page cached.

If the product insists on one scrolling list, I would **virtualise** it with something like `react-window` so only the visible rows are in the DOM — 500 rows becomes about 20 nodes. And I would memoise the row component so changing a filter does not re-render every row."

Memory angle, if they probe: 500 mounted rows means 500 component instances plus their DOM nodes held in the heap. Virtualising drops that to the visible window, which is why it helps memory as much as speed.

Prevent: *"A performance budget on that page, and I would test with realistic data volumes — this is the classic bug that passes testing with ten rows and fails in production."*

S2 • "A user fills in 9 screens of the Business Card form, presses back, and loses everything."

This is your own feature, so answer it as a design question you already solved.

"That is almost certainly state scoped to the wrong component. If each step owns its own state, navigating away unmounts the step and the state is destroyed with it.

The way I built it, **all the form data lives in the parent wizard component** and each step is a presentational component receiving values and an onChange. Moving between steps unmounts the step but not the parent, so nothing is lost.

If the requirement is surviving a **refresh or a closed tab**, that is a different problem — in-memory state cannot survive that. Then you persist: either to `localStorage` on each step, or better for this case, save a draft server-side so it works across devices. On Business Card the application had a real status, so an incomplete one was a persisted draft the client could return to — which is also what the My Requests 'Incomplete' tab showed."

That last connection is strong — two of your features explaining each other.

S3 "The notification bell shows 3 unread, but the user has already read them."

Causes: the count is cached and never refetched · it is fetched once on mount and the app is a long-lived SPA · the read action updates the server but not the local state · two components hold separate copies of the count.

"First I would check whether the server is right — if the API returns 3, it is a backend problem; if it returns 0 and the UI shows 3, it is stale client state.

For the client case, the fix is a single source of truth. The count lives in one place in Redux, and the action that marks something read updates it. I would also refetch on window focus, because an SPA left open on a second tab goes stale, and that is exactly when users notice."

Prevent: "Avoid two components deriving the same number independently — that is how they drift."

S4 • "Your session-timeout popup fires while users are actively typing."

Your own feature again, and a nice judgement question.

"That means the timer is measuring time since **login** rather than time since **last activity**. The fix is to reset the countdown on real user activity — keypress, click, scroll, and any API call going out.

Two details matter. I would **throttle** the activity handler, because resetting a timer on every mousemove is wasteful. And the client timer must stay aligned with the **server's** session expiry — if the server expires the session at 30 minutes, a client clock that has drifted will either warn too late or keep someone 'logged in' on a dead session. The server is the source of truth; the popup is only a courtesy."

Prevent: "And the cleanup matters — if the listeners and the interval are not removed on unmount, you leak the component and can end up with several timers running at once."

S5 • "A user types 'axis' in a search box. Results flash, then the wrong results appear."

"That is a **race condition**. Typing fires a request per keystroke; 'ax' and 'axis' are both in flight, and if the older one answers last it overwrites the newer results.

Two fixes, and I would use both. **Debounce** the input by about 400ms, so nine keystrokes become one request instead of nine. And **cancel the previous request** with an `AbortController` in the `useEffect` cleanup, so a stale response can never land.

That cleanup solves a second problem at the same time — without it you also get the 'state update on an unmounted component' warning when the user navigates away mid-request."

Prevent: "Handle all four states — loading, error, empty and success. An empty dropdown with no message is the thing that makes it look broken."

S6 "Typing one character in a form re-renders 50 components."

"I would open the **React DevTools Profiler**, record one keystroke and see what rendered and why — it tells you whether it was props, state or the parent.

The usual cause is that the input's state lives too high up, so every keystroke re-renders the whole page. Three fixes, in order: **move the state down** into the smallest component that needs it; wrap expensive children in `React.memo` ; and if the parent passes callbacks, wrap them in `useCallback` , because a new function each render is a new object in memory and `React.memo` compares by reference, so it would never match.

On a large form I would also consider React Hook Form, which keeps fields uncontrolled — that is what we used on the 14-screen Business Card flow so the whole form did not re-render per keystroke."

S7 "After a release the Lighthouse score dropped from 88 to 70. Find out why."

"First, compare like with like — same page, same network throttling, ideally the same machine, because Lighthouse varies run to run.

Then I would look at what the release changed. The usual suspects are a new dependency inflating the bundle, an image added without dimensions, or a lazy-loaded route that is now imported eagerly — one static import can pull a whole chunk back into the main bundle, which quietly undoes code-splitting.

I would run the bundle analyser before and after. And I would check which metric moved: LCP points at an image or the server, CLS at a layout shift from a missing dimension, INP at new JavaScript blocking the main thread."

Prevent: *"Lighthouse in CI with a budget, so the build flags the regression instead of someone noticing weeks later."*

Part B · Backend scenarios

S8 ●● "An API is fast in testing and takes 8 seconds in production. Why?"

The single most likely backend scenario for your profile.

Clarify: Is it slow for all requests or some? Did it change after a release, or degrade gradually?

"Gradual degradation with no code change points at **data volume** — something that was fine with a hundred rows is not fine with a hundred thousand.

I would find out where the time goes before changing anything: application timings or logs first to confirm it is the database, then SQL logging to count the queries.

The most likely cause is **N+1** — one query loads the list and then each row triggers its own query for a relation. A hundred rows becomes a hundred and one round trips, and in testing with five rows you never notice. The fix is a join fetch, an `@EntityGraph`, or best for a read-only screen, a **DTO projection** that selects only the needed columns.

If it is one slow query rather than many, I would run `EXPLAIN` — a full table scan on a column in the `WHERE` clause means a missing index."

The cost, which you should volunteer: *"An index speeds up reads but every insert and update has to maintain it, so I add them deliberately on a write-heavy table rather than by default."*

Memory angle: *"N+1 is also a memory problem — every one of those queries puts entities plus their snapshots into the persistence context, so you fill the heap with objects a screen may need three fields from."*

Prevent: *"Test with production-like data volumes, and log a warning when a request exceeds a threshold so it surfaces before a client reports it."*

S9 ●● "Two approvers open the same card application and click Approve at the same moment."

A perfect question for your Camunda approval work, and it is really about concurrency.

"Both transactions read the status as PENDING, both decide it is approvable, and both write APPROVED. So it is approved twice and you get two approval records. That is a **lost update** — and on a financial workflow it is an audit problem, not just a data problem.

The fix I would reach for first is **optimistic locking** — a `@Version` column on the entity. Hibernate includes the version in the update's `WHERE` clause, so the second write affects zero rows and throws `OptimisticLockException`. I catch that and tell the second approver the application has already been actioned, and show them the current state.

The alternative is **pessimistic locking**, `SELECT ... FOR UPDATE`, which makes the second transaction wait. I would avoid it here because it holds a database lock, and genuine collisions are rare — optimistic costs nothing when there is no conflict."

The follow-up to pre-empt: *"A double-click by one user is the same shape of bug. That one I would also guard with idempotency — the same request key should not create a second approval."*

S10 • "A client says they downloaded a report, but it is not in your audit log."

This is a trap question built straight from your AOP system — and the best answer is one almost nobody gives.

"The first thing I would check is whether the method was called **through the proxy**. Spring AOP works by wrapping the bean in a proxy, so if some other method inside the same class called the download method directly with `this.download(...)`, the call never goes through the proxy and the aspect never fires. That is the same caveat as `@Transactional`, and it is the most likely cause of a silently missing audit record.

After that: does the **pointcut expression** actually match that method — a new endpoint added in a different package would not be picked up. And is the audit insert failing and being swallowed somewhere, which would make the audit silently incomplete rather than noisily broken."

Prevent, and this is the strong part:

"An audit trail you cannot trust is worse than none, because you believe it. So I would want the audit write to fail loudly, or go onto a queue so a database problem cannot lose it, and an alert if the audit volume drops unexpectedly — for a system writing 50,000 a day, a sudden drop is the signal that something stopped matching."

S11 "Your audit logging starts slowing down the APIs it audits."

"That means the audit write is happening **inside** the request. At 50,000 a day that is fine; at ten times that, the insert is on the critical path of every call.

I would move it off the request: publish the audit event to a queue and let a consumer write it, so the API returns as soon as the business work is done. We had RabbitMQ and Kafka on the platform, so that path existed.

The trade-off is honest though — it becomes **eventually consistent**, and if the consumer is down the audit lags. For compliance data that is a decision to take with the business, not alone. So: a durable queue, retries, a dead-letter queue, and alerting on it. An async audit with no DLQ just moves the failure somewhere nobody is looking."

A cheaper middle option to mention: batching inserts rather than one per call.

S12 • "You need to add an approval step to the Camunda workflow, but 200 applications are in flight."

"The problem is that in-flight process instances are running the old definition. If I just deploy a new one, the running instances either carry on with the old flow or, worse, end up in an inconsistent state.

Camunda handles this with **versioned process definitions** — deploying creates a new version, and existing instances keep running the version they started on. So the safe default is: new applications get the new flow, the 200 in flight finish on the old one.

If the business needs the existing ones migrated — say the new step is a regulatory requirement — that is a deliberate **process instance migration**, mapping old activities to new ones, and I would do it in a lower environment first with a plan to roll back."

The judgement line: *"The first question I would ask is which of those two the business actually wants, because it is a business decision and the technical work is different."*

S13 • "Add a column to a table a live application is using. How do you deploy it safely?"

"With **Liquibase**, which we used, so the change is a versioned changeset reviewed like any other code rather than someone running SQL by hand.

The rule is to make it **backwards compatible**, because for a window the old code and the new schema run together. So: add the column as **nullable** or with a default, deploy the schema change first, then deploy the code that uses it. Never add a NOT NULL column with no default to a populated table — it fails or locks.

Removing a column is the same idea in reverse: stop writing to it, deploy, then drop it in a later release once nothing references it. Expand first, contract later."

Prevent: *"And the changeset needs to be tested on a copy with real data volumes — a migration that takes two seconds on an empty table can lock a large one for minutes."*

S14 "A third-party payment or document service you call starts timing out."

Your FileNet integration makes this credible.

"The immediate danger is not that one call fails — it is that every thread waiting on that call piles up until the service runs out of threads and stops serving requests it could have answered. One slow dependency takes the whole service down.

So, first: **every network call needs a timeout**. A call with no timeout is the actual bug. Then a **circuit breaker** — after a failure threshold it opens and fails fast instead of waiting, and after a cool-down it lets a test call through. Failing fast protects both sides: my threads stay free, and the struggling service gets room to recover.

Then the question is what to do in the fallback. For a document download, degrade gracefully — tell the user it is temporarily unavailable. For something that must not be lost, queue it and retry.

And retries only on **idempotent** operations. Retrying a payment blindly can charge someone twice."

S15 "You need to send 50,000 notification emails for a platform announcement."

Your notification work makes this a natural question.

"Not in a request, and not in a loop. I would publish the work to a queue and have consumers process it, which gives me three things: the triggering request returns immediately, the work survives a restart, and I can scale consumers.

Then the practical constraints: the email provider will **rate limit**, so I throttle rather than firing 50,000 at once. Each message needs to be **idempotent** — keyed on something like user plus announcement id — because at-least-once delivery means a consumer can see the same

message twice, and sending a customer the same email twice is a visible failure.

And I would track delivery status so support can answer 'did they get it?', with failures going to a dead-letter queue rather than disappearing."

Part C · Judgement and teamwork scenarios

Round 2 is explicitly about fitment. These matter more than people expect.

S16 • "QA raises a bug you cannot reproduce."

"I assume it is real — 'works on my machine' is not a finding. Usually the difference is data, environment or sequence, so I ask for the exact steps, the account used, the environment and the time, then look at the logs for that window rather than trying to guess.

If I still cannot reproduce it, I pair with the tester and watch them do it. That has found it more often than anything else, because the step they did not think worth mentioning is usually the one that matters.

If it is genuinely intermittent, I add logging around the suspect path and wait for it to happen again, rather than guessing at a fix. A fix for a bug you have not reproduced is a guess you have shipped."

S17 "You are halfway through a sprint and realise your estimate was badly wrong."

"Say it immediately, at the next stand-up at the latest. The cost of a slipped estimate is small; the cost of finding out on the last day is that nobody could re-plan around it.

I would come with specifics rather than just 'it is bigger' — what I found, how much more I think it is, and options: cut scope, split it so the core ships this sprint, or take the extra time. That makes it a decision someone can take rather than a problem handed over.

Then I would look at what I missed in the estimate, because that is usually integration or testing, which is the part people forget."

S18 "A reviewer leaves a comment you disagree with."

"I assume they can see something I cannot, so I ask before I argue — often there is context about the codebase I do not have.

If I still disagree after understanding it, I say so with a reason, not a preference: a case it breaks, or a concrete cost. And I separate 'this is wrong' from 'I would have done it differently' — the second is not worth blocking over.

If we are still apart after two or three exchanges, I stop typing and go and talk to them. Long review threads are usually a misunderstanding rather than a real disagreement."

S19 • "You find a security problem in code already in production."

"Escalate immediately — this is the one case where I would not sit on it to investigate first. I tell my lead straight away, because the decision about disclosure and timing is not mine to make.

Then: how exposed is it, has it been exploited, can we mitigate quickly even before a proper fix. And I would be careful where I write the details — a public ticket describing exactly how to exploit something unpatched is its own problem.

In a banking context there is usually a defined process for this, and following it matters more than being fast on my own."

S20 "You join navAjna and get a legacy codebase with no tests and no documentation."

A very likely question, given the JD says you work across multiple client projects.

"I would not start by refactoring. First I read — trace one important request end to end, from the controller through the service to the database, because that teaches you the conventions faster than reading files at random.

I would get it running locally and change something small and low-risk, to prove I can build, test and deploy it.

Then I would add tests **as I go** rather than as a project — when I touch an area, I write a test for the behaviour I am about to change, so it is pinned before I change it. Trying to retro-fit full coverage up front never gets finished and nobody thanks you for it.

And I would write down what I learn as I go, because the gaps are obvious to me in week one and invisible by week six."

S21 "A teammate's PR works, but you think the approach is wrong."

"It depends on how wrong. If it works and is just not how I would have done it, that is my preference and I would let it go, or mention it as a non-blocking comment.

If I think it will cause a real problem — a performance issue at scale, something that will be hard to change later — I say so, with the specific scenario where it breaks rather than an opinion. And I would offer to pair on it rather than leaving a wall of comments, because a ten-minute conversation beats a twenty-comment thread."

S22 "A client stakeholder asks you directly for a change, bypassing the process."

Worth having, because the JD mentions client stakeholders.

"I would not say no, and I would not just do it. I would understand what they actually need — sometimes the ask is a solution and the real need is different.

Then I take it back through the proper channel — my lead or the BA — so it gets prioritised and does not quietly displace committed work. What I would not do is silently absorb it, because then the sprint slips and nobody knows why.

The tone matters though. 'Let me check how we fit that in and come back to you today' lands very differently from 'you need to raise a ticket'."

✓ The five scenarios to rehearse out loud

If you only prepare five, make them these — they are the most likely and the most revealing:

1. **S8** — the API that is fast in testing and slow in production (*N+1, your strongest ground*)
2. **S9** — two approvers at once (*concurrency, from your own workflow*)
3. **S10** — the missing audit record (*the AOP proxy answer almost nobody gives*)
4. **S1** — the slow list page (*your Lighthouse work, reframed as a problem to solve*)
5. **S20** — the legacy codebase with no tests (*very likely, given their multi-project model*)

And remember step 5 every time: what you would add so it does not happen again.

06 · Walk-In Checklist and One-Page Recall

Read the night before, and again in the auto on the way.

Carry these — the JD lists them explicitly

- [] **Updated resume** — 3 printed copies
- [] **Government photo ID** — Aadhaar or PAN
- [] **Education certificates** — B.Tech degree and marks memos
- [] Payslips / offer letter / relieving letter from Zensar, in case they ask
- [] Pen and a small notebook
- [] Phone charged, with your GitHub and portfolio reachable — **it is face to face, so you can actually show your work**
- [] Water

Address: Level 3 & 4, Karan Arcade, Plot #78, Patrika Nagar, HITEC City, Hyderabad 500081.

Leave early. HITEC City traffic is unpredictable and arriving flustered costs you the first ten minutes. Both rounds are the same day, so expect to be there a while — eat first.

The 10 answers that carry this interview

1. **HashMap** — `hashCode()` → spread `h ^ (h >>> 16)` → bucket `(n-1) & hash`; collisions chain, over 8 entries becomes a red-black tree; capacity 16, load factor 0.75, resizes at 12.
 2. **equals/hashCode** — equal objects must have equal hash codes, or the map looks in the wrong bucket and never finds the entry.
 3. **Re-renders** — own state · props · **parent re-rendered** · context. Fix with `React.memo` → `useCallback` / `useMemo` → move state down.
 4. **key** — stable identity. Index breaks on delete or reorder; React reuses the wrong DOM node and the typed value stays on the wrong row.
 5. **Four states** — loading, error, **empty**, success. Empty is the one people forget.
 6. **fetch does not throw on 404 or 500** — check `res.ok` and throw yourself.
 7. **JWT** — header.payload.signature, **signed not encrypted**, nothing sensitive inside. Authorisation must be server-side; a hidden button is not security.
 8. **N+1** — one query for the list, one per row for the relation. JOIN FETCH, `@EntityGraph`, batch fetch, or a DTO projection.
 9. **@Transactional** / **AOP proxy** — an internal `this.method()` call bypasses the proxy, so nothing fires.
 10. **Microservices** — independent deploy, scale, fault isolation; **cost** is network failure, no distributed transaction, harder debugging. *Say the cost.*
-

Memory — the backbone answer •

Stack = one per thread, holds method frames and local variables, cleared automatically when the method returns. **Heap** = one, shared, holds every object, cleaned by the GC.

Primitives hold the value. Objects hold a reference — an address into the heap.

```
void change(int n, StringBuilder s) { n = 99; s.append(" world"); }  
// n unchanged (copied the value) · s changed (copied the address)  
// -> Java is always pass-by-value; for objects the value IS the address
```

- **StackOverflowError** = recursion with no base case · **OutOfMemoryError** = too many live objects, usually something unbounded.
- **A Java memory leak** = keeping a reference you no longer need, so the GC is not allowed to collect it.
- **ArrayList** = one array, compact, cache friendly · **LinkedList** = a separate node per item with two pointers, ~3x the overhead, scattered.
- **HashMap holds** a table array + **one Node per entry** + deliberately empty slots (0.75 load factor). Resizing allocates a new array **twice the size** while the old one still exists.
- **String pool** — literals are shared, `new String()` is a separate object. `a == b` true, `a == c` false.
- **+ in a loop** — new String object every iteration, $O(n^2)$ and garbage. Use `StringBuilder`.
- **Integer cache** -128..127. `int` is a stack value; `Integer` is a heap object.
- **Spring singleton** — one object in the heap for all requests; locals are per-request stack, **instance fields are shared**.
- **JPA persistence context** keeps the entity **plus a snapshot** for dirty checking — two copies per row, which is why batch jobs OOM. DTO projections avoid it.
- **React:** mutating state keeps the same reference so no re-render; a new function each render is a new heap object, which is why `useCallback` exists; **an uncleaned timer or listener holds the whole component alive** — that is what a React leak is.

Scenario questions — the five-step method •

Clarify → Diagnose → Causes → Fix + cost → Prevent.

Most candidates jump to the fix. The marks are in **diagnose** (say how you would find out before saying what you would change) and **prevent** (a test, a log, an alert, so it cannot recur). Almost nobody does the last one.

The line to reuse: *"Before I change anything I would want to know where the time is actually going — my first guess is usually wrong."*

The five most likely, all from your own work:

- **Slow in prod, fast in testing** → $N+1$. Count the queries in the SQL log. Join fetch, `@EntityGraph`, or a DTO projection. *Index cost: slows writes.*
- **Two approvers at once** → lost update. `@Version` optimistic locking; the second write

hits zero rows and you tell them it is already actioned.

- **Missing audit record** → the **AOP proxy**: an internal `this.method()` call never goes through the proxy, so the aspect never fires. Same caveat as `@Transactional`.
 - **Slow list page** → server-side pagination first; virtualise if it must be one list; memoise the row.
 - **Legacy codebase, no tests** → read one request end to end, make one small safe change, then add tests **as you touch things**, not as a project.
-

| Your numbers — say them without hesitating

20,000+ corporate clients · **Lighthouse 62 → 88** · **60%** off card processing ·
50,000+ audit transactions a day · **1,000+** monthly reports · **14-screen** onboarding flow · **7-screen** recall workflow · **3+** features reusing your components ·
40% fewer re-renders on QKart · Core Web Vitals **65 → 82** at Pixelvide.

| The three sentences that decide this

Opening: *"...and I am available to join immediately, so the seven-day window is not a problem for me."* **Say this in the first two minutes.**

The gap: two sentences, the real reason, then **specifically** what you have done since February — Crio.Do, anything you have built, the GenAI certifications. Then pivot back to availability.

Track: *"Frontend primarily — React, Redux and TypeScript are my day-to-day. But I built the Spring Boot backends for the same applications, so I can work across both."*

| In the room

- **Structure your answers.** "Three things —" then give three.
 - **Say your answer and stop.** Over-explaining is the commonest nervous habit.
 - **Think out loud in the coding question.** Silence reads as stuck.
 - **Ask the edge cases before you code.** Null, empty, duplicates.
 - **State the complexity before they ask.**
 - **If you do not know it:** *"I have not used that directly — my understanding is X. Is that the direction you mean?"* Never bluff; the second question always comes.
 - **Ask them questions.** Round 2 is fitment; make it a conversation.
-

| The last five minutes before you walk in

You have shipped five production features on a banking platform used by 20,000 clients, both ends of the stack, with real numbers attached. For a role asking 1–3 years, that is a strong profile — and you can start next week, which most of the queue cannot.

Sit up. Breathe out slowly. Smile before you speak.

Go get it.